

```
In [10]: df = DataFrame(A = 1:5, B = ["M", "F", "F", "M", "F"])
```

```
Out[10]: 5 rows x 2 columns
```

	A	B
	Int64	String
1	1	M
2	2	F
3	3	F
4	4	M
5	5	F

Figure 1: Simple DataFrame

Here we are building the DataFrame using two columns — A and B. Column A holds values from one to five, and column B has 'M' and 'F' as values. The output is shown in Figure 1.

Once the DataFrame is built, the individual columns can be accessed with the help of column names. For example:

```
df.A
5-element Array{Int64,1}:
 1
 2
 3
 4
 5
```

DataFrames.jl supports both symbol representation and string representation for column names. The string representation of the column name is given within quotes, as shown below:

```
df."A"
5-element Array{Int64,1}:
 1
 2
 3
 4
 5
```

The individual column names can be fetched using the `names()` function:

```
2-element Array{String,1}:
 "A"
 "B"
```

If you want the same thing to be represented as a symbol, use the `propertynames()` function:

```
propertynames(df)
2-element Array{Symbol,1}:
 :A
 :B
```

Passing a single value

Julia allows the passing of a single value to a column, as shown in the following example:

```
df = DataFrame("Magazine" => "OSFY",
               "Theme" => ["Mobile", "Cloud", "Top10",
                           "Web"])
print(df)
```

The output is shown below:

```
4x2 DataFrame
```

Row	Magazine	Theme
	String	String
1	OSFY	Mobile
2	OSFY	Cloud
3	OSFY	Top10
4	OSFY	Web

Column by column approach

In DataFrames, columns can be added individually:

```
df = DataFrame()
```

```
df.Month_Id = 1:12
df.Month_Name = ["Jan", "Feb", "Mar",
                 "Apr", "May", "Jun", "Jul", "Aug",
                 "Sep", "Oct", "Nov", "Dec"]
print(df)
```

The output is shown below:

```
12x2 DataFrame
```

Row	Month_Id	Month_Name
	Int64	String
1	1	Jan
2	2	Feb
3	3	Mar
4	4	Apr
5	5	May
6	6	Jun
7	7	Jul
8	8	Aug
9	9	Sep
10	10	Oct
11	11	Nov
12	12	Dec

The size of the DataFrame may be fetched with the `size()` function:

```
size(df)
(12, 2)
```

Row by row approach

The DataFrame can be built in a row by row manner as well. The row can be added with the `push()` function, as shown below:

```
dframe = DataFrame{A = Int[], B = String[]}
push!(dframe, (1, "January"))
push!(dframe, (1, "February"))
print(dframe)
```

The output is:

```
2x2 DataFrame
```

Row	A	B
	Int64	String
1	1	January
2	1	February